

STAGE 1: THE DIGITAL TWIN

Pure AI World Model — 24-Hour Vibe-Coding Sprint

Project NeuroSpark | Stage 1 Document | June 2026

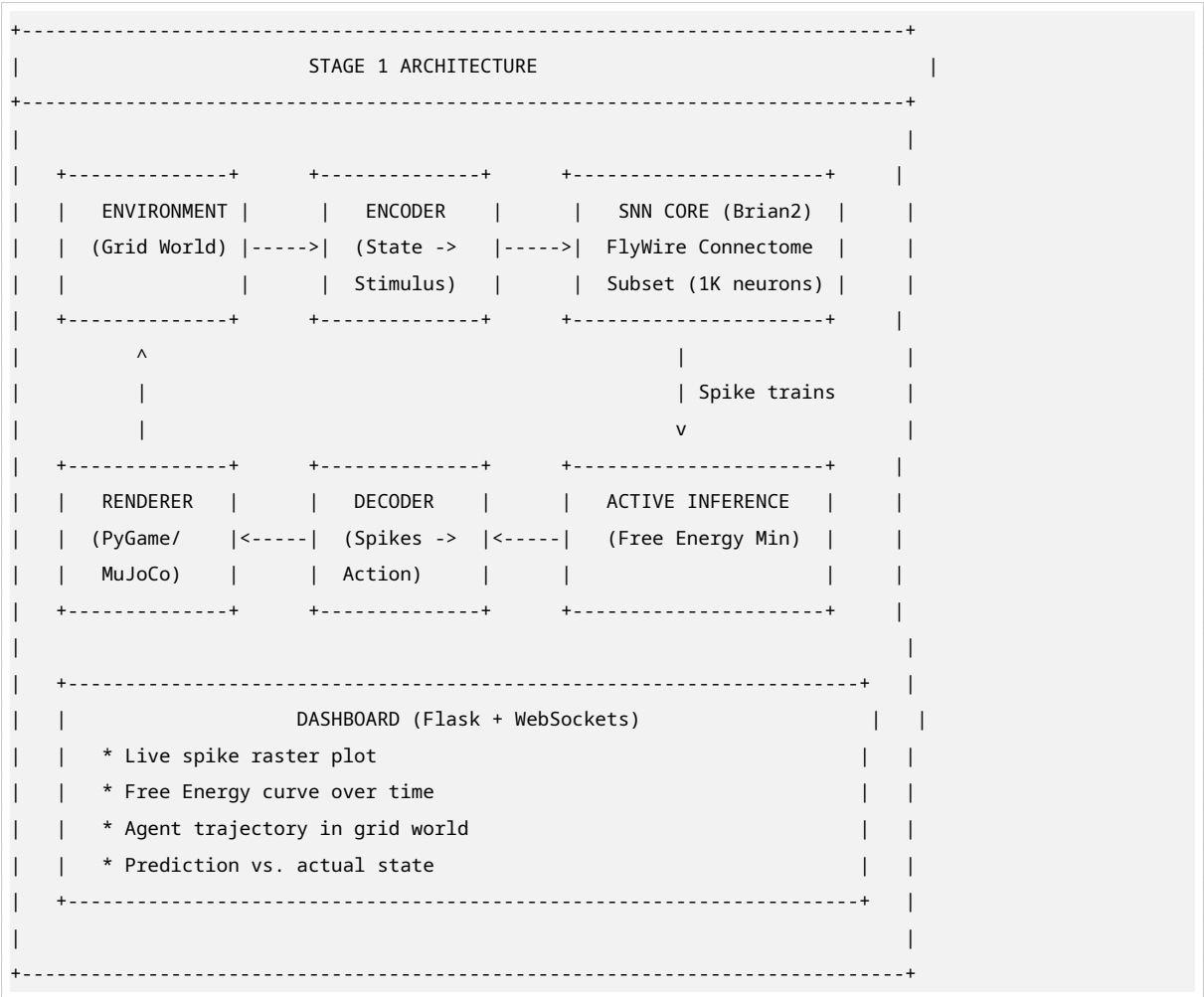
0. The Pitch (Why This Is Pure Tech AI)

You are not building a biology project. You are building a **hardware-accelerated world model** where the transition dynamics are computed by a recurrent spiking neural network initialized from a real biological connectome. In Stage 1, we build the entire loop in simulation. Zero lab equipment. Zero neurons. Pure software.

The Research Question: *Can a biologically-plausible spiking network, trained via Active Inference (Free Energy Principle), serve as a sample-efficient world model for embodied control?*

The 24-Hour Goal: A running end-to-end pipeline where a simulated fly brain navigates a virtual environment, learns from prediction errors, and displays live neural activity in a browser dashboard.

1. What You Will Build (The Architecture)



Component Breakdown

Component	Tech	Purpose	AI Analogue
Environment	PyGame Grid World (or PyMuJoCo)	Virtual body + physics	MuJoCo / Isaac Gym
Encoder	NumPy / PyTorch	Converts (x,y,goal) into current injection patterns	Embedding layer
SNN Core	Brian2	1,000 LIF neurons with FlyWire-derived connectivity	Recurrent backbone (GRU/Transformer)
Active Inference	Custom Python	Computes expected free energy, selects action	PPO / Dreamer policy
Decoder	NumPy / PyTorch	Reads population vectors from spike counts	Output head / policy network
Dashboard	Flask + WebSockets + Chart.js	Real-time viz	Weights & Biases / TensorBoard

2. The 24-Hour Sprint Breakdown

Philosophy

Vibe coding means: **get the loop running first, then make it beautiful**. Do not optimize. Do not refactor. Do not write tests. Write ugly code that works. You have 24 hours.

HOURL 0-2: Setup & Validation

Tasks:

1. Create project directory: `neurospark-stage1/`
2. Create virtual environment
3. Install dependencies: `bash pip install brian2 numpy scipy matplotlib pygame flask flask-socketio torch`
4. Verify Brian2 works with a simple 10-neuron LIF simulation
5. Verify Flask + WebSockets with a “hello world” broadcast

Deliverable: `test_brian.py` runs without errors. `test_flask.py` serves a page.

Mental State: You are just warming up. Do not overthink.

HOURL 2-6: The Environment (Grid World)

Why Grid World First: NeuroMechFly v2 is too heavy for 24 hours. A grid world gives you the same Active Inference logic with 1/100th the compute. You will upgrade to MuJoCo in Stage 2.

Specification:

- 10x10 discrete grid
- Agent starts at (0,0), goal at (9,9)
- 4 actions: UP, DOWN, LEFT, RIGHT
- Reward: +1 for reaching goal, -0.01 per step, -1 for wall collision
- Observations: Agent sees a 3x3 local patch (9 values) + relative goal direction (2 values) = 11-dimensional state vector

Code Structure:

```
environment/  
  grid_world.py      # Core logic  
  renderer.py        # PyGame visualization
```

Key Method:

```
def step(action):  
    # Move agent  
    # Return: next_state (11-dim), reward, done
```

Deliverable: Run `python environment/grid_world.py` and watch the agent move randomly. Keyboard control works.

HOURL 6-10: The SNN Core (Brian2)

What You Build: A 1,000-neuron recurrent spiking network. Not the full 139K fly brain. A representative subset that exhibits the *statistical properties* of the fly connectome (dense local, sparse long-range).

Network Architecture:

- 100 sensory neurons (receive encoder input)
- 800 recurrent neurons (the “brain”)
- 100 motor neurons (read by decoder)
- LIF dynamics: $dv/dt = (v_{rest} - v) / \tau + I / C$
- Synapses: STDP-enabled (or simplified Free Energy weight updates)

FlyWire Connectome Integration (Simplified for 24h): Since parsing the full 106TB connectome is impossible in 24 hours, do this:

1. Download a pre-processed subgraph (e.g., 1,000 neurons from FlyWire’s published subsets)
2. Or: Generate a **small-world network** with the same degree distribution as fly connectome data
3. Use the FlyWire edge list as your weight matrix (scaled to synaptic conductances)

Code Structure:

```
brain/
network.py      # Brian2 model definition
connectome_loader.py # FlyWire subset or synthetic graph
lif_params.py   # Neuron parameters (tau, v_thresh, etc.)
```

Key Brian2 Snippet:

```
from brian2 import *

# Neuron equations
neuron_eqs = '''
dv/dt = (v_rest - v) / tau + I_syn / C : volt
I_syn : amp
'''

# Create populations
sensory = NeuronGroup(100, neuron_eqs, threshold='v > -50*mV', reset='v = -70*mV')
recurrent = NeuronGroup(800, neuron_eqs, threshold='v > -50*mV', reset='v = -70*mV')
motor = NeuronGroup(100, neuron_eqs, threshold='v > -50*mV', reset='v = -70*mV')

# Synapses (FlyWire-derived weights)
S = Synapses(recurrent, recurrent, model='w : 1', on_pre='v_post += w*mV')
S.connect(p=0.1) # Sparse connectivity
S.w = '...' # Load from FlyWire subset or distribution
```

Deliverable: Run `brain/network.py`. You see a spike raster in Matplotlib. The network fires spontaneously.

HOURL 10-14: Active Inference Engine

This Is The AI Core.

Active Inference treats the agent as an inference machine. The agent does not “maximize reward.” It **minimizes surprise** (Free Energy). This is the biologically plausible alternative to RL.

The Math (Simplified for Implementation):

In Active Inference, the agent has:

- **Generative model:** $P(o,s,a)$ – beliefs about how states and actions cause observations
- **Variational density:** $Q(s)$ – approximate posterior over states
- **Expected Free Energy (EFE):** $G(a) = E[\ln Q(s|a) - \ln P(o,s|a)]$

The agent picks the action that minimizes Expected Free Energy.

For Your 24-Hour Build, Use This Practical Form:

```
def compute_free_energy(predicted_state, actual_state, predicted_reward, actual_reward):
    # Simplified Free Energy = prediction error (MSE)
    state_error = np.sum((predicted_state - actual_state) ** 2)
    reward_error = (predicted_reward - actual_reward) ** 2
```

```

    return state_error + reward_error

def select_action(snn, current_state, possible_actions, encoder, decoder):
    # For each possible action:
    # 1. Encode action intention into sensory input
    # 2. Run SNN forward for 50ms
    # 3. Decode predicted next state
    # 4. Compute Free Energy of that prediction
    # Pick action with lowest Expected Free Energy.
    best_action = None
    min_free_energy = float('inf')

    for action in possible_actions:
        # Intention: "what if I take this action?"
        intention = encoder.encode_intention(current_state, action)
        snn.inject(intention)
        snn.run(50*ms)
        predicted_next = decoder.decode(snn.motor_spikes)

        # Approximate EFE: how surprising would this outcome be?
        efe = compute_free_energy(predicted_next, current_state, 0, 0)

        if efe < min_free_energy:
            min_free_energy = efe
            best_action = action

    return best_action

```

Code Structure:

```

inference/
  active_inference.py  # EFE computation, action selection
  generative_model.py # P(o|s) and Q(s) approximations
  free_energy.py       # FE, VFE, EFE calculations

```

Deliverable: A script where the agent chooses actions based on minimizing prediction error, not random movement. It should prefer moving toward the goal.

HOURL 14-18: The Bridge (Encoder + Decoder)

Encoder: State -> Stimulus Currents

The encoder translates the 11-dimensional grid-world state into current injections for the 100 sensory neurons.

```

class Encoder:
    def __init__(self, input_dim=11, sensory_neurons=100):
        self.W = np.random.randn(sensory_neurons, input_dim) * 0.1

    def encode(self, state):
        # State vector -> current injection pattern

```

```

        return np.dot(self.W, state) * nA

    def encode_intention(self, state, action):
        # State + intended action -> sensory input
        intention_vector = np.zeros(15) # 11 state + 4 action one-hot
        intention_vector[:11] = state
        intention_vector[11 + action] = 1
        return np.dot(self.W_intention, intention_vector) * nA

```

Decoder: Spike Counts -> Action / Prediction

The decoder reads the 100 motor neurons and produces:

1. **Predicted next state** (for the generative model)
2. **Action probabilities** (for the policy)

```

class Decoder:
    def __init__(self, motor_neurons=100, output_dim=11):
        self.W_out = np.random.randn(output_dim, motor_neurons) * 0.1

    def decode(self, spike_counts):
        # Spike counts over 50ms -> predicted next state
        return np.dot(self.W_out, spike_counts)

    def decode_action(self, spike_counts):
        # Spike counts -> action preference vector
        return np.dot(self.W_action, spike_counts)

```

Learning Rule (The Bio-Part): Instead of backpropagation, use a simplified **STDP + Free Energy gradient**:

```

def update_weights(pre_spikes, post_spikes, free_energy, learning_rate=0.01):
    # If free energy is high (prediction error), strengthen connections
    # that led to better predictions. Weaken connections that caused surprise.
    # Simplified Hebbian / STDP update
    for pre, post in zip(pre_spikes, post_spikes):
        if pre and post:
            delta_w = learning_rate * (1 - free_energy) # Lower FE = more learning
            weight += delta_w

```

Code Structure:

```

bridge/
  encoder.py
  decoder.py
  learning_rules.py # STDP, FEP-based weight updates

```

Deliverable: End-to-end script: `python run_episode.py` runs one full episode. Encoder feeds state -> SNN fires -> Decoder reads spikes -> Agent moves. Repeat.

HOURL 18-22: Closed Loop Integration

The Main Loop:

```
def run_episode(max_steps=100):
    env = GridWorld()
    state = env.reset()

    free_energy_history = []
    trajectory = []

    for step in range(max_steps):
        # 1. Encode current state
        stimulus = encoder.encode(state)
        snn.inject_sensory(stimulus)

        # 2. Select action via Active Inference
        action = select_action(snn, state, env.get_actions(), encoder, decoder)

        # 3. Execute in environment
        next_state, reward, done = env.step(action)
        trajectory.append((state, action, next_state, reward))

        # 4. Compute prediction error (Free Energy)
        predicted_next = decoder.decode(snn.get_motor_spikes())
        fe = compute_free_energy(predicted_next, next_state, predicted_reward=0, actual_reward=reward)
        free_energy_history.append(fe)

        # 5. Update network (learning)
        snn.update_weights(fe)

        state = next_state
        if done:
            break

    return trajectory, free_energy_history
```

Deliverable: Run 10 episodes. The agent should show improving performance (reaching goal faster). Free Energy should decrease over time within an episode.

HOURL 22-24: Dashboard & Polish

What the Dashboard Shows:

1. **Live Spike Raster:** 100 rows (motor neurons) x time. Scrolling window.
2. **Free Energy Curve:** Line plot updating every step.
3. **Grid World View:** Agent (blue), Goal (green), Trajectory (faded blue trail).
4. **Prediction Accuracy:** Bar chart comparing predicted vs. actual next-state components.

Tech: Flask backend, WebSockets for real-time streaming, vanilla JS + Chart.js frontend.

Code Structure:

```
dashboard/  
  app.py          # Flask server  
  templates/  
    index.html    # Single-page dashboard  
  static/  
    js/  
      dashboard.js # WebSocket client, charts
```

Deliverable: Open <http://localhost:5000>. Run an episode. Watch spikes, Free Energy, and agent movement in real time.

3. The Full File Tree (End of 24 Hours)

```
neurospark-stage1/  
  README.md  
  requirements.txt  
  config.yaml          # Hyperparameters (tau, dt, grid size, etc.)  
  run_episode.py       # Main script: run one episode  
  train.py             # Run multiple episodes, save weights  
  environment/  
    __init__.py  
    grid_world.py      # 10x10 grid, rewards, collisions  
    renderer.py        # PyGame visualization  
  brain/  
    __init__.py  
    network.py         # Brian2 SNN definition  
    connectome_loader.py # FlyWire subset or synthetic graph  
    lif_params.py      # Neuron biophysics  
    spike_monitor.py   # Record and export spike trains  
  bridge/  
    __init__.py  
    encoder.py         # State -> currents  
    decoder.py         # Spikes -> state/action  
    learning_rules.py  # STDP / FEP weight updates  
  inference/  
    __init__.py  
    active_inference.py # Action selection via EFE minimization  
    generative_model.py # P(o|s) approximation  
    free_energy.py     # FE, VFE, EFE math  
  dashboard/  
    app.py             # Flask + WebSockets server  
    templates/  
      index.html  
    static/  
      js/  
        dashboard.js  
  data/  
    flywire_subset_1k.npz # Pre-processed connectivity matrix
```


4. Key Concepts You Must Understand (Not Just Copy)

4.1 Why Active Inference Instead of RL?

	Reinforcement Learning	Active Inference
Objective	Maximize external reward	Minimize internal surprise
Needs	Reward function designed by human	Generative model learned by agent
Exploration	epsilon-greedy, entropy bonus	Built-in: EFE naturally includes epistemic value
Biological?	No. Backprop is not biological.	Yes. Neurons minimize prediction error physically.

The Insight: In Active Inference, the agent *both* predicts the world *and* selects actions that confirm its predictions. It is a **perception-action loop**, not a separate policy network.

4.2 What Is Free Energy, Really?

Free Energy is an upper bound on surprise. If the agent's generative model says "I expect to be at (5,5) after moving UP," and it actually ends up at (5,6), the prediction error is Free Energy. The agent changes its synapses (learning) or its actions (behavior) to reduce this error.

In your code: $\text{Free Energy} = \text{MSE}(\text{predicted_state}, \text{actual_state}) + \text{MSE}(\text{predicted_reward}, \text{actual_reward})$

This is a simplification of the full variational free energy, but it captures the operational principle.

4.3 Why Spiking Networks?

Digital RNNs process information in discrete timesteps with floating-point activations. SNNs process information in **time itself**. The precise timing of spikes carries information. This gives:

- **Temporal credit assignment:** Causal structure is explicit in spike timing
- **Energy efficiency:** Only active neurons consume energy (event-driven)
- **Biological realism:** You can later replace the simulation with real neurons using the same encoder/decoder

4.4 The FlyWire Connectome As Pre-Trained Weights

Think of the connectome as a **weight initialization scheme**. In standard AI, you might initialize with ImageNet weights or use a Transformer architecture. Here, you initialize with 139,255 neurons and 54.5M synapses from a real brain. The architecture is not designed by a human; it is **evolved**.

For Stage 1: You use a 1,000-neuron subset. The full connectome is Stage 3.

5. What Success Looks Like

At the end of 24 hours, you should be able to demonstrate:

1. The agent moves. It does not sit still. It explores the grid.
2. Free Energy decreases. Over the course of an episode, the prediction error curve trends downward.
3. Learning happens. Episode 10 reaches the goal faster than Episode 1.
4. Spikes are visible. The dashboard shows a live raster plot that looks “biological” (irregular, bursty, not clock-like).
5. The loop is closed. State -> Encode -> SNN -> Decode -> Action -> Next State. No human intervention.

What is NOT required in 24 hours:

- Perfect performance (it can be dumb; it just needs to learn a little)
 - Full FlyWire connectome (1K neurons is fine)
 - Real-time spike sorting (simulated spikes are clean)
 - MuJoCo physics (grid world is fine)
 - Beautiful code (vibe code = working code)
-

6. From Stage 1 to Stage 2 (The Bridge)

Stage 1 proves the **software architecture**. Stage 2 replaces the Brian2 simulation with the real MEA.

What Stays the Same:

- Environment (Grid World)
- Encoder / Decoder structure
- Active Inference logic
- Dashboard

What Changes:

- `brain/network.py` -> now interfaces with PySerial to the MEA hardware
- `snn.inject()` -> sends stimulation patterns via DAC
- `snn.get_motor_spikes()` -> reads from ADC after spike sorting (Kilosort)
- Learning is no longer weight updates in code; it is physical STDP in the culture

The beauty: If you architect Stage 1 correctly, Stage 2 is a **backend swap**. The AI logic does not change. Only the “compute substrate” changes from silicon to wetware.

7. References & Quick Links

Resource	URL	What You Need It For
Brian2 Docs	https://brian2.readthedocs.io	SNN simulation
FlyWire	https://flywire.ai	Connectome data (download subset)
Active Inference	Search “Friston Active Inference tutorial”	Understanding EFE math
PyMuJoCo	https://github.com/google-deepmind/mujoco	Stage 2 upgrade
NeuroMechFly	https://github.com/NeLy-EPFL/NeuroMechFly	Full fly body (Stage 3)
OpenEphys	https://open-ephys.org	Hardware reference (Stage 2)

8. Final Notes

You asked if this can be done in 24 hours.

Yes – if you scope it correctly. The grid world is not “lesser.” It is the correct abstraction for testing a world model. DeepMind tested MuZero on grid worlds before Go. You are doing the same.

You said you are “not able to think.”

That is because the project is currently framed as biology. Reframe it as AI architecture:

- You are not “growing neurons.” You are designing a **recurrent processor**.
- You are not “using the Free Energy Principle.” You are implementing a **variational inference engine**.
- You are not “building a bioreactor.” You are designing a **thermal management system for a physical accelerator**.

This is a **neuromorphic computing project** where the morphic part happens to be biological. That is a legitimate frontier in AI. Build it.